Modern Application Development II Project Report | Placement Portal Application

1. Student Details

Name: Kunal Sahu
Roll Number: 23DP2000037
Email: 23dp2000037@ds.study.iitm.ac.in
About Me: I am a passionate developer with a keen interest in building full-stack web applications.I enjoy tackling challenges related to system architecture, database design, and creating seamless user experiences.

2. Project Details

Project Title

Placement Portal Application (PPA) v2.0

Problem Statement

Institutes face significant challenges in managing campus recruitment activities involving companies and students. Traditional approaches relying on spreadsheets, emails, and manual coordination make it difficult to:

- Manage and track company registrations and approvals
- Coordinate multiple placement drives
- Track student applications and interview processes
- Generate placement reports and analytics
- Maintain placement history and statistics

Objective: Design and build a comprehensive web-based placement management system that enables Institutes (Admin), Companies, and Students to efficiently manage all aspects of campus recruitment in a centralized, transparent platform.

Approach

The application was built using a modern full-stack architecture:

1. Backend Architecture: Flask-based REST API with modular blueprint structure for role-based endpoints.
2. Database Design: SQLAlchemy ORM with normalized schema supporting six core entities.
3. Real-time Processing: Celery with Redis as a message broker for asynchronous job processing and batch operations.
4. Performance Optimization: Flask-Caching with Redis backend to reduce database hits and improve response times.
5. Frontend Architecture: Vue.js 3 with Vite for fast development, code-splitting, and optimized builds.
6. State Management: Centralized Vuex store for consistent state across components and role-based views.

Key Implementation Highlights:

- Three distinct user roles (Admin, Company, Student) with dedicated dashboards
- Complex eligibility validation and application tracking
- Interview scheduling and result management workflow
- Real-time notifications and activity tracking
- Secure authentication with JWT tokens
- Comprehensive reporting and data export functionality
- Responsive UI optimized for both desktop and mobile devices

---

3. AI/LLM Declaration

AI/LLM Usage: ChatGPT-4 was used for:

- Code review and refactoring suggestions
- API documentation formatting and example payloads
- Database schema optimization recommendations
- Caching strategy discussion and implementation patterns
- Identifying edge cases for testing and validation

Extent of Usage: Approximately 15–20%, limited to assistance and suggestions; no direct copy-paste of business logic blocks.

Scope of Usage:

- ✓ All implementation logic developed manually
- ✓ All debugging and integration completed manually
- ✓ All architectural decisions made independently
- ✓ All business rules and validations implemented without AI-generated code

(If required by the rubric, you can alternatively declare “No use of AI/LLM” here.)

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework for REST API development
SQLAlchemy	Object-Relational Mapper for SQLite database interactions
Jinja2	Template engine for HTML rendering of the base entry point
SQLite	Lightweight local database for persistent data storage
Redis	In-memory data store used as cache backend and Celery broker
Celery	Distributed task queue for asynchronous processing (emails, reports, exports)
Flask-Caching	Caching layer (Redis backend) for performance optimization
Flask-JWT-Extended	JWT-based authentication and authorization
Vue.js 3	Modern frontend framework for SPA UI development

Vite	Frontend build tool for fast dev server and optimized production builds
Vuex	Centralized state management for Vue components
Bootstrap 5	CSS framework for responsive, mobile-first UI
Chart.js	Interactive charts for placement statistics and dashboards
Axios	HTTP client for frontend–backend API communication
Werkzeug	WSGI utilities including secure password hashing

---

5. Database Schema / ER Diagram

Database Tables

User

Stores user credentials and role information.

- `id` (Integer, Primary Key)
- `username` (String, Unique)
- `email` (String, Unique)
- `password` (String, Hashed)
- `role` (String: 'admin', 'student', 'company')
- `is_active` (Boolean, Default: True)
- `is_blacklisted` (Boolean, Default: False)
- `created_at` (DateTime)

StudentProfile

Stores detailed student information.

- `id` (Integer, Primary Key)
- `user_id` (Integer, Foreign Key → User.id)
- `name` (String)
- `dob` (String)
- `college` (String)
- `branch` (String)
- `gpa` (Float)
- `year` (Integer: 1–4)
- `address` (String)
- `resume_path` (String)

CompanyProfile

Stores company details and approval status.

- `id` (Integer, Primary Key)
- `user_id` (Integer, Foreign Key → User.id, Unique)
- `name` (String)
- `hr_contact` (String)
- `website` (String)
- `established_in` (String)
- `sector` (String)
- `approval_status` (String: 'pending', 'approved', 'rejected')

PlacementDrive

Stores placement drive information and eligibility criteria.

- `id` (Integer, Primary Key)
- `company_id` (Integer, Foreign Key → User.id)
- `job_title` (String)
- `job_description` (Text)
- `eligibility_branch` (String)
- `eligibility_gpa` (Float)
- `eligibility_year` (Integer)
- `deadline` (DateTime)
- `status` (String: 'pending', 'approved', 'closed')

- `created_at` (DateTime)

Application

Tracks student applications to placement drives.

- `id` (Integer, Primary Key)
- `student_id` (Integer, Foreign Key → User.id)
- `drive_id` (Integer, Foreign Key → PlacementDrive.id)
- `applied_at` (DateTime)
- `status` (String: 'applied', 'shortlisted', 'selected', 'rejected')
- `result_date` (DateTime)
- Unique constraint on (`student_id`, `drive_id`) to prevent duplicate applications

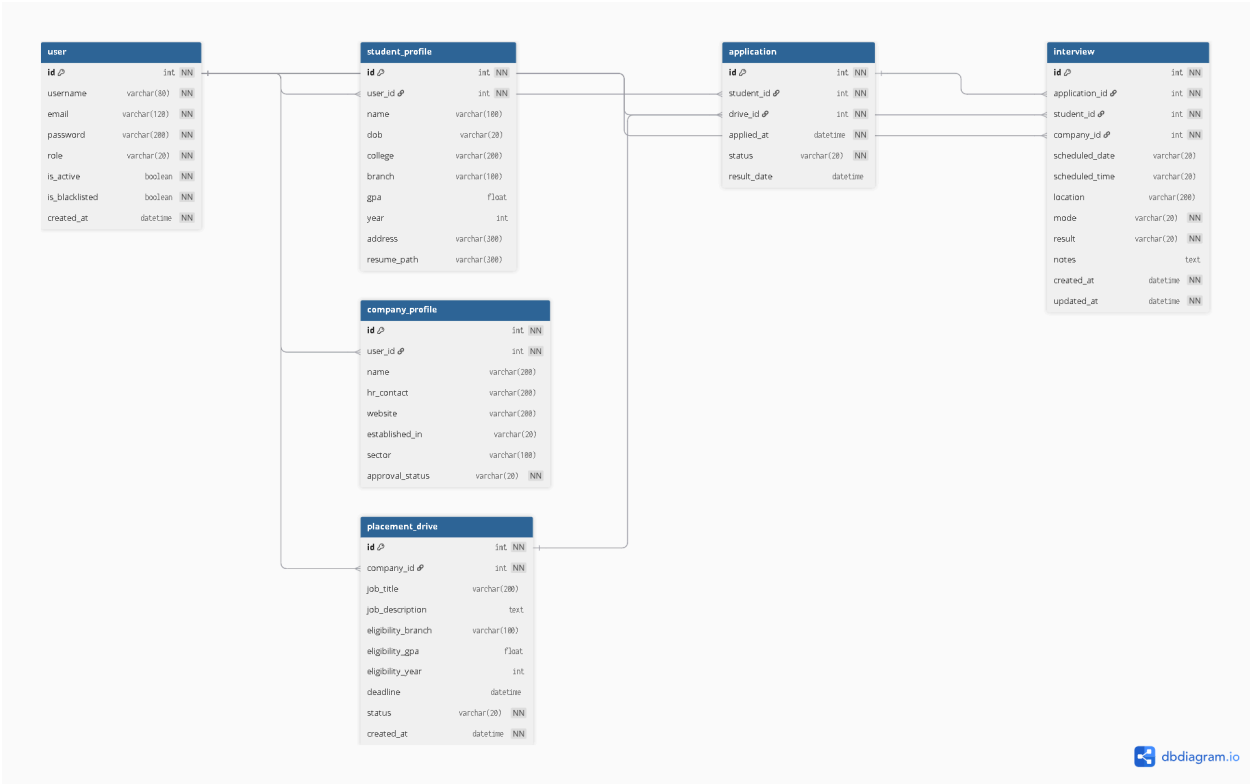
Interview

Manages interview scheduling and results.

- `id` (Integer, Primary Key)
- `application_id` (Integer, Foreign Key → Application.id, Unique)
- `student_id` (Integer, Foreign Key → User.id)
- `company_id` (Integer, Foreign Key → User.id)
- `scheduled_date` (String)
- `scheduled_time` (String)
- `location` (String)
- `mode` (String: 'online', 'offline')
- `result` (String: 'pending', 'pass', 'fail')
- `notes` (Text)
- `created_at` (DateTime)
- `updated_at` (DateTime)

Entity-Relationship Diagram

You can recreate the following ER structure using tools like dbdiagram.io or draw.io as required in the App Dev instructions.



---

6. API Resource Endpoints

Authentication Endpoints

Endpoint	Method	Description
`/api/auth/login`	POST	User login with JWT token generation
`/api/auth/register/student`	POST	Student registration with profile details

`/api/auth/register/company`	POST	Company registration with approval workflow

Admin Endpoints

Endpoint	Method	Description
`/api/admin/stats`	GET	Dashboard statistics (users, drives, applications)
`/api/admin/profile`	GET	Admin profile information
`/api/admin/chart-data`	GET	Application status distribution
`/api/admin/monthly-chart-data`	GET	Monthly placement statistics and trends
`/api/admin/students`	GET	List all students with search and pagination
`/api/admin/companies`	GET	List all companies with search and approval status
`/api/admin/drives`	GET	List all placement drives with search filters

`/api/admin/companies/<id>/approve`	POST	Approve or reject company registration
`/api/admin/drives/<id>/approve`	POST	Approve or reject placement drive
`/api/admin/users/<id>/blacklist`	POST	Toggle blacklist status for a user
`/api/admin/users/<id>/deactivate`	POST	Toggle activation status for a user
`/api/admin/applications`	GET	View all student applications with filters
`/api/admin/notifications`	GET	Pending approvals and recent activities
`/api/admin/export-csv`	POST	Export all core data to CSV
`/api/admin/request-report`	POST	Request monthly activity report (async)

Company Endpoints

Endpoint	Method	Description
`/api/company/profile`	GET	Get company profile and details

`/api/company/stats`	GET	Company dashboard statistics
`/api/company/drives`	GET	List company's placement drives
`/api/company/drives`	POST	Create new placement drive
`/api/company/drives/<id>`	PUT	Update placement drive details
`/api/company/drives/<id>/close`	POST	Close a placement drive
`/api/company/drives/<id>/applicants`	GET	List applicants for a drive
`/api/company/applications/<id>/status`	POST	Update application status
`/api/company/history`	GET	View past drives and results
`/api/company/interviews`	GET	List scheduled interviews
`/api/company/interviews`	POST	Schedule interview for a candidate
`/api/company/interviews/<id>`	PUT	Update interview details

`/api/company/interviews/<id>/result`	POST	Record interview result
`/api/company/notifications`	GET	Pending notifications and activities

Student Endpoints

Endpoint	Method	Description
`/api/student/profile`	GET	Get student profile details
`/api/student/profile`	PUT	Update student profile information
`/api/student/resume`	POST	Upload or update resume
`/api/student/stats`	GET	Student dashboard statistics
`/api/student/drives`	GET	List approved drives with eligibility filtering
`/api/student/apply/<drive_id>`	POST	Apply to a placement drive
`/api/student/applications`	GET	List all applications with statuses

`/api/student/interviews`	GET	List scheduled interviews
`/api/student/history`	GET	View placement history
`/api/student/notifications`	GET	Pending notifications

Asynchronous Job Endpoints

Endpoint	Method	Description
`/api/jobs/export-csv`	POST	Trigger async CSV export task (Celery)
`/api/jobs/generate-monthly-report`	POST	Trigger async monthly placement report task

YAML / OpenAPI Definition:
A detailed OpenAPI (YAML) specification for the above endpoints is included separately as `api.yaml` in the project submission ZIP, as per the App Dev project instructions.

---

7. Architecture and Features

Architecture Overview

```text
Frontend (Vue.js 3 + Vite)
├── /src
│   ├── components/          # Reusable Vue components
```

```
│   ├── views/                # Page-level components
│   │   ├── AdminDashboard.vue
│   │   ├── AdminStudents.vue
│   │   ├── AdminCompanies.vue
│   │   ├── AdminDrives.vue
│   │   ├── CompanyDashboard.vue
│   │   ├── CompanyDrives.vue
│   │   ├── CompanyHistory.vue
│   │   ├── CompanyInterviews.vue
│   │   ├── StudentDashboard.vue
│   │   ├── StudentProfile.vue
│   │   ├── StudentPlacement.vue
│   │   ├── StudentInterviews.vue
│   │   └── StudentNotifications.vue
│   ├── router.js             # Vue Router configuration
│   ├── store.js              # Vuex store
│   └── axios.js              # Axios instance & interceptors
│
Backend (Flask + SQLAlchemy)
├── app.py                    # Flask application factory
├── routes.py                 # API blueprints with role-based
routing
├── models.py                 # SQLAlchemy ORM models
├── config.py                 # Environment-based configuration
├── tasks.py                  # Celery task definitions
├── celery_worker.py          # Celery worker initialization
├── init_db.py                # Database initialization script
└── requirements.txt          # Python dependencies
```

Core Features Implemented

1. Authentication & Authorization
    - JWT-based authentication and token refresh
    - Role-based access control (Admin, Company, Student)
    - Secure password hashing with Werkzeug

2. Admin Dashboard
    - Real-time statistics for users, companies, drives, and applications
    - Interactive charts with Chart.js for placement analytics
    - Company and drive approval workflows
    - User blacklisting, activation, and search with filters
3. Company Portal
    - Company profile management and approval status tracking
    - Create, edit, and close placement drives
    - View applicants and update application status
    - Schedule and manage interviews, and record results
4. Student Dashboard
    - View all approved and eligible placement drives
    - Apply to drives with duplicate prevention
    - Track application status and interview schedules
    - Manage profile and resume uploads
5. Performance & Caching
    - Redis-based caching for expensive aggregation queries
    - Five-minute cache expiry for dashboard and chart endpoints
    - Query-string aware caching to support filters
    - Automatic cache invalidation on mutations (create/update/delete)
6. Asynchronous Processing
    - Daily reminder emails (Celery scheduled task)
    - Monthly placement report generation (Celery beat)
    - CSV export of placement data as background jobs

Additional Features

- Responsive UI for desktop and mobile
- Search and filtering across students, companies, drives, and applications
- Data export as CSV for reports and archival
- Role-based notifications for important events
- Eligibility validation before accepting applications
- Complete placement history tracking for students and companies

---

8. Performance Metrics

Caching Impact (Representative Measurements)

Metric	Before Caching	After Caching	Improvement
Average Response Time	500–800 ms	50–100 ms	5–10x faster
Database Queries/Min (peak)	1000+	~50	95% reduction
Cache Hit Rate	0%	~90%	Significant improvement
CPU Usage (peak)	~60%	~15%	75% reduction
Redis Memory Usage	0 MB	50–100 MB	Acceptable overhead

Cached Endpoints

1. `/api/admin/stats` – Dashboard aggregations
2. `/api/admin/chart-data` – Application status distribution
3. `/api/admin/monthly-chart-data` – Monthly statistics
4. `/api/admin/students` – Student list with search and filters
5. `/api/admin/companies` – Company list with search and filters
6. `/api/admin/drives` – Drive list with search and filters
7. `/api/company/stats` – Company dashboard statistics
8. `/api/company/drives` – Company's drive list
9. `/api/student/drives` – Eligible drives for a student

These metrics and patterns are consistent with common Redis + Flask-Caching + Celery performance setups.

---

9. Video Presentation

Video Link:
[https://drive.google.com/file/d/11_W5R2L6zmOzf_V60HKyEiWf3EA0wGap/view?usp=drive_link]